Python Engineering - Test Runner Function References

TESTS, test vs test(), PASS / FAIL / ERROR y jerarquía de excepciones

1. La idea central: una función también es un objeto

En Python una función no es solo algo que podemos ejecutar. También es un objeto que podemos guardar en una variable, meter en una lista o pasar como argumento.

```
def test_initial_state():
    ...

function_reference = test_initial_state
# no parentheses: the function is NOT executed yet
```

La variable `function_reference` apunta a la función. Todavía no la hemos llamado.

2. Diferencia entre test y test()

Expresión	Significado
test	Referencia a la función. No se ejecuta.
test()	Llamada a la función. Se ejecuta ahora.

```
test_initial_state    # function object / reference
test_initial_state()  # execute the function
```

Los paréntesis son la señal de llamada. Esta diferencia es la misma que vimos anteriormente entre `add` y `add(...)`.

3. ¿Por qué creamos una lista TESTS?

```
TESTS = [
    test_initial_state,
    test_euler_step,
    test_csv_round_trip,
    test_missing_file_raises,
]
```

Guardamos referencias a funciones para que el runner pueda tratarlas todas de la misma forma. La lista no guarda resultados; guarda **funciones que se podrán ejecutar después**.

Esto permite construir un único algoritmo genérico para cualquier cantidad de tests.

4. ¿Qué pasaría si escribimos paréntesis dentro de TESTS?

```
# Not what we want here:
TESTS = [
    test_initial_state(),
    test_euler_step(),
]
```

En ese caso las funciones se ejecutarían inmediatamente mientras Python crea la lista. La lista almacenaría sus valores de retorno, que normalmente serían `None`, no las funciones.

5. ¿Qué significa for test in TESTS?

```
for test in TESTS:
    ...
```

En cada iteración, la variable `test` pasa a referenciar una función diferente.

```
iteration 1:
test = test_initial_state

iteration 2:
test = test_euler_step

iteration 3:
test = test_csv_round_trip

iteration 4:
test = test_missing_file_raises
```

El nombre `test` es solo una variable local del bucle. Podría llamarse `function`, aunque 'test' comunica mejor la intención.

6. ¿Por qué ponemos test() dentro del try?

```
for test in TESTS:
    try:
        test()
```

Porque en ese punto sí queremos ejecutar la función que está referenciada por `test`.

```
test = test_initial_state
test()
# equivalent to:
test_initial_state()
```

En la siguiente iteración, exactamente la misma línea `test()` ejecutará otra función. El runner no necesita conocer sus detalles internos.

7. Ventaja del patrón

Si mañana añadimos un nuevo test:

```
def test_specific_energy():
    ...
```

solo añadimos su referencia a la lista:

```
TESTS = [
    test_initial_state,
    test_euler_step,
    test_csv_round_trip,
    test_missing_file_raises,
    test_specific_energy,
]
```

El bucle y la lógica PASS / FAIL / ERROR permanecen sin cambios.

8. Las funciones son objetos: conexión con Class 10

```
TESTS = [function_A, function_B, function_C]

for function in TESTS:
    function()
```

El runner trabaja con funciones como datos: las almacena, las recorre y finalmente las llama.

9. El flujo general del test runner

```
for test in TESTS:
    try:
        test()
    except AssertionError as error:
        # FAIL
        ...
    except Exception as error:
        # ERROR
        ...
    else:
        # PASS
        ...
```

Hay tres resultados conceptualmente distintos: PASS, FAIL y ERROR.

10. PASS

Si `test()` termina sin lanzar ninguna excepción, el bloque `else` se ejecuta.

```
test()
   |
   +-- no exception --> else --> PASS
```

PASS significa que el test pudo ejecutarse y sus assertions se cumplieron.

11. FAIL: AssertionError

Cuando una assertion es falsa, Python lanza `AssertionError`.

```
result = np.array([102.0, 19.019, 0.1])
expected = np.array([999.0, 19.019, 0.1])

assert np.allclose(result, expected)
# -> AssertionError
```

El código ha llegado hasta la comprobación, pero el resultado observado no coincide con la expectativa. Por eso el runner lo clasifica como **FAIL**.

12. ERROR: otra excepción inesperada

Si el test encuentra un problema distinto de una assertion fallida, por ejemplo:

```
float('hello')        # ValueError
[10, 20][50]          # IndexError
10 / 0                # ZeroDivisionError
missing_dict['x']     # KeyError
```

el test no ha completado normalmente su camino de verificación. El runner lo clasifica como **ERROR**.

13. Exception no significa 'error de contenido'

`Exception` es una clase base amplia que agrupa muchos tipos de excepciones habituales.

```
Exception
|-- ValueError
|-- TypeError
|-- FileNotFoundError
|-- IndexError
|-- KeyError
|-- ZeroDivisionError
`-- AssertionError
```

Por tanto, el segundo `except Exception` significa aproximadamente: 'si ocurre otra excepción normal que no hemos clasificado antes, regístrala como ERROR'.

14. AssertionError también es una Exception

Este punto es muy importante: `AssertionError` también pertenece a la familia `Exception`.

```
AssertionError
    is an Exception
```

Entonces, ¿por qué no entra siempre en `except Exception`? Porque Python evalúa los `except` en orden y usa el primer bloque compatible.

15. El orden de los except importa

```
except AssertionError as error:
    # specific -> FAIL
    ...

except Exception as error:
    # general -> ERROR
    ...
```

Primero colocamos la excepción específica; después la general.

Si lo hiciéramos al revés:

```
except Exception as error:
    # catches AssertionError too
    ...

except AssertionError as error:
    # would never classify it here
    ...
```

un `AssertionError` sería capturado por el bloque general y aparecería como ERROR en lugar de FAIL.

16. Regla general de orden

```
specific exception first
        |
        v
more general exception later
```

Esta regla no es exclusiva del testing: es una regla útil para cualquier cadena de except.

17. ¿Por qué usamos Exception aquí pero no en EX23?

En EX23 conocíamos los fallos concretos que queríamos gestionar:

```
FileNotFoundError
ValueError
```

Por tanto era mejor capturarlos específicamente.

En `run_tests.py`, el objetivo del nivel exterior es distinto: queremos que un error inesperado en un test no impida ejecutar los tests restantes.

```
test 1 -> PASS
test 2 -> TypeError
test 3 -> should still run
test 4 -> should still run
```

Por eso el runner tiene una captura general al final.

18. Capturar Exception no significa ocultar el error

El runner captura la excepción, pero muestra su tipo y mensaje:

```
except Exception as error:
    print(f'ERROR {test.__name__}')
    print(f'{type(error).__name__}: {error}')
```

El error sigue siendo visible. Simplemente evitamos que termine todo el runner antes de ejecutar el resto de pruebas.

19. Ejemplo completo de clasificación

Situación	Excepción	Clasificación
Todos los assert son True	Ninguna	PASS
assert result == expected es False	AssertionError	FAIL
float('hello')	ValueError	ERROR
acceso a índice inexistente	IndexError	ERROR
archivo inesperadamente ausente	FileNotFoundError	ERROR

20. Resumen final

```
TESTS
-> list of function references

for test in TESTS
-> test refers to a different function
   on each iteration

test
-> function reference

test()
-> execute that function

AssertionError
-> FAIL
-> test reached its check,
   but the expectation was not satisfied

another Exception
-> ERROR
-> something unexpected happened
   while executing the test

no exception
-> PASS
```